

智慧校园系统 - 开发环境搭建指南

项目基本信息

- 项目名称: 智慧校园系统登录应用 (SGTPDILoginApp)
- 项目类型: ASP.NET Core Web 应用
- 前端技术: HTML, Tailwind CSS
- 后端技术: C#, .NET 8.0, .NET MAUI
- 开发工具: Visual Studio 2026

核心步骤列表

环境搭建概览

- 需求分析阶段: 明确项目技术栈与版本要求
- 环境搭建阶段: 安装配置 Visual Studio 2026 与依赖
- 项目初始化阶段: 使用命令行创建项目并配置版本控制
- 测试验证阶段: 构建项目, 运行并验证前后端功能
- 部署运维阶段: 代码提交与远程推送, 总结常见问题

开发工具清单

序号	开发工具	主要用途	版本要求
1	.NET SDK	框架运行与编译环境	8.0
2	Visual Studio 2026	核心 IDE 开发环境	2026
3	Git	代码版本控制	最新版
4	Tailwind CSS	前端页面样式优化	最新版

☒ 成功标准

- ☐ Visual Studio 2026 安装并成功配置 .NET MAUI 工作负载
- ☐ .NET 8.0 环境配置完成
- ☐ ASP.NET Core 项目成功创建并可编译
- ☐ 成功运行 `dotnet run` 并在本地端口 (如 5046) 访问
- ☐ 登录页面功能正常 (输入学号 2023210643, 密码 123456 成功登录)
- ☐ 成功跳转并展示带有 "Hello World!" 和学生信息的页面
- ☐ 项目代码成功提交并推送至 Gitee 远程仓库分支

第一课时：基础环境搭建（1 学时）

1.1 Visual Studio 与 .NET 环境配置

步骤 1：下载与安装 Visual Studio 2026

- 访问微软官方网站下载 Visual Studio 2026 安装程序。
- 运行安装程序，在工作负载选择界面，务必勾选 ".NET MAUI" 相关工作负载。
- 等待安装完成并启动 Visual Studio。

步骤 2：验证 .NET 环境

- 打开命令行工具（CMD 或 PowerShell）。
- 执行命令：`dotnet --version`
- 确认输出版本号包含 8.0，证明 .NET 8.0 SDK 环境准备就绪。

第二课时：项目初始化与配置（1 学时）

2.1 创建 ASP.NET Core 项目

步骤 1：初始化 Web 应用

- 打开命令行工具，进入目标工作区。
- 执行项目创建命令：`dotnet new webapp -o SGTPDILoginApp`
- 进入生成的项目目录：`cd SGTPDILoginApp`

2.2 Git 初始化与远程仓库关联

步骤 1：配置本地与远程仓库

- 在项目根目录执行初始化：`git init`
- 关联 Gitee 远程仓库：`git remote add origin https://gitee.com/chzucz1dl/cg2-sgtpdi.git`
- 创建并切换到专属开发分支：`git checkout -b SGTPDI-C#+.NET+MAUI`

第三课时：功能实现与页面设计（1 学时）

3.1 登录功能开发

步骤 1：编写登录页面

- 创建或修改 `Index.cshtml` 页面，实现包含学号和密码输入框的表单。
- 引入 Tailwind CSS 优化页面样式，去掉默认的 bar 栏和 footer 栏，实现现代化布局。
- 增加交互效果，如按钮悬停效果和输入框焦点效果。

步骤 2：实现登录逻辑

- 打开 `Index.cshtml.cs` 文件。
- 编写登录验证逻辑，设定测试账号：`2023210643`，密码：`123456`。

3. 添加错误提示机制，输入错误时在前端显示提示信息。

3.2 学生信息页面开发

步骤 1: 编写信息展示与退出功能

1. 创建 `HelloWorld.cshtml` 页面。
2. 页面中展示学生学号 (2023210643)、姓名 (张海天) 以及 "Hello World!" 欢迎信息。
3. 在 `HelloWorld.cshtml.cs` 中实现退出登录逻辑，点击退出后重定向至登录页面。

第四课时：功能验证与问题排查 (1 学时)

4.1 项目本地部署与验证

步骤 1: 构建与运行项目

1. 在项目目录执行构建命令: `dotnet build`
2. 构建成功后，执行运行命令: `dotnet run`
3. 观察控制台输出，确认服务监听的本地端口 (例如 `http://localhost:5046`) 。

步骤 2: 系统功能测试

1. 登录测试: 浏览器访问项目地址，输入正确的账号密码，验证是否成功跳转到学生信息页面。测试错误密码是否正确拦截并提示。
2. 信息展示测试: 确认 `HelloWorld.cshtml` 页面正常显示学号和姓名。
3. 退出测试: 点击退出登录按钮，验证是否成功返回登录页。

4.2 常见问题排查

问题 1: 执行 dotnet run 提示端口被占用

- **原因:** 默认端口被其他程序占用。
- **解决:** 打开 `Properties/launchSettings.json`，修改 `applicationUrl` 的端口号后重新运行。

问题 2: Tailwind CSS 样式未生效

- **原因:** CDN 链接未引入或本地缓存问题。
- **解决:** 检查 `_Layout.cshtml` 的 `<head>` 标签，确保正确引入了 Tailwind 样式表。

问题 3: Git 推送失败

- **原因:** 本地分支落后于远程分支。
- **解决:** 先执行 `git pull origin SGTPDI-C#+.NET+MAUI` 合并代码，再进行 push。

总结与思考

实验总结

- 掌握基于 C# 和 .NET 8.0 的 Web 应用开发环境搭建
- 熟悉 Visual Studio 2026 和 .NET 命令行工具的使用
- 了解 Git 版本控制及 Gitee 远程仓库的基本操作
- 完成项目的创建、功能实现、本地运行和代码推送

课后思考

28. ASP.NET Core 的 Razor Pages 模式在开发小型 Web 应用时有哪些效率优势？
29. 如何进一步通过依赖注入（DI）和实体框架（EF Core）将硬编码的账号密码改为连接数据库验证？
30. .NET MAUI 的跨平台特性对本项目未来向 Android/iOS 端扩展有什么帮助？
-

附录：快速启动命令

项目运行

```
# 进入项目目录
cd SGTPDILoginApp

# 编译项目
dotnet build

# 运行项目
dotnet run
```

Git 提交与推送

```
git add .
git commit -m "update: 优化项目结构"
git push origin SGTPDI-C#+.NET+MAUI
```